Datasets & Environment Summary

Cập nhật 2026-10-29 (branch `hmm-navigation`). Hướng nghiên cứu hiện tại: HMM sequence-based visual localization; đề bài và việc tiếp theo ở `handoff/00_READ_ME_FIRST.md`. Kết quả audit dữ liệu (Bước 1): `DATA_AUDIT.md`. Dataset mới cho hướng HMM (chuỗi dài + odometry thật): §1b 4Seasons dump và §1c Boreas bên dưới.

1. Dataset quan trọng nhất: descriptor dumps (PUBLIC)

`nhatlong1103/vpr-descriptor-dumps`, link https://www.kaggle.com/datasets/nhatlong1103/vpr-descriptor-dumps (public, 4.4 GB, giấy phép CC BY-NC-SA 4.0). Chỉ có **global descriptor** (không có local feature), nên đủ cho HMM mức place nhưng chưa đủ cho ước lượng 6DOF.

Đây là vector đặc trưng (global descriptor, float16, đã chuẩn hoá L2) của **mọi ảnh map và query** trong 4 benchmark, tính bằng 6 mạng VPR, kèm pose, điều kiện và tên ảnh. Mọi thí nghiệm SelfCal trước đây chỉ cần dataset này (không cần ảnh gốc). Tên ảnh chứa timestamp, nên `vprlib.frame_order` dựng lại được thứ tự chuỗi.

Khi thêm acc Kaggle mới: KHÔNG upload lại. Chỉ cần thêm dataset này vào kernel:

- kernel-metadata: `"dataset_sources": ["nhatlong1103/vpr-descriptor-dumps", "<acc>/dive-vpr-analysis"]`
- Hoặc trên giao diện notebook: **Add Input → Datasets → tìm "vpr-descriptor-dumps"**.
- Khi mount, nó nằm ở `/kaggle/input/datasets/nhatlong1103/vpr-descriptor-dumps/dump/` (hoặc `/kaggle/input/vpr-descriptor-dumps/dump/`). Runner của `automation/make_eval_kernel.py` tự tìm thư mục `meta/`.

Cấu trúc thư mục

```
dump/
  salad/     (8448-d)  SALAD: DINOv2-B + optimal-transport aggregation
  megaloc/   (8448-d)  MegaLoc
  mixvpr/    (4096-d)  MixVPR, ResNet-50
  netvlad/   (4096-d)  NetVLAD, VGG-16 + PCA-whitening 4096
  cosplace/  (2048-d)  CosPlace, ResNet-50
  dinov2/    (1536-d)  DINOv2-B: CLS + mean patch token
  meta/                pose / condition / image names
README.md
```

Mỗi thư mục descriptor có cùng 32 file. Thứ tự hàng trùng với file `meta` tương ứng.

File	Dataset	Số ảnh (map / query)
`rc_ref.npy`, `rc_q.npy`	RobotCar-Seasons v2: reference map chính thức + query train công khai (9 điều kiện)	6,954 / 1,906
`cmu_s<N>_{db,q}.npy`	Extended-CMU-Seasons, 11 slice chính thức N ∈ {6, 7, 8, 10, 11, 12, 13, 21, 22, 23, 24} (gộp thành một bản đồ thành phố 2,418 ảnh)	ví dụ s6: 272 / 1,463
`nord_<season>.npy` (spring, summer, fall, winter)	Nordland, 4 mùa căn khung 1:1 (subsample)	4,500 mỗi mùa
`msls_{cph,sf}_{db,q}.npy`	MSLS val: Copenhagen / San Francisco (bỏ panorama)	12,556 / 6,559 và 6,315 / 4,525

File trong `meta/`:

File	Khoá
`rc.npz`	`ref_names, ref_C` (tâm camera), `ref_R, ref_focal, q_names, q_C, q_R, q_cond`. Điều kiện: dawn, dusk, night, night-rain, overcast-summer, overcast-winter, rain, snow, sun

File	Khoá
`cmu_s*.npz`	`db_names, db_C, db_R, q_names, q_C, q_R, q_cond` (ngày chụp = điều kiện)
`nord.npz`	`frame_ids, seasons`
`msls_{cph,sf}.npz`	`db_keys, db_C` (UTM east/north), `db_ca` (compass), `db_view, db_date, q_keys, q_C, q_ca, q_view, q_date, q_cond` (city-season, cph-night)

Loader: `analysis/vprlib.py`, dùng `V.LOADERS[ds](dump_dir, descriptor)` với `ds` ∈ `rc, cmu, nord, msls, nordw` (`nordw` = Nordland lấy mùa đông làm map).

Nguồn gốc dump: do các kernel GPU của acc `nhatlong1103` tạo ra: `dive-vpr-nb15a/b/c/d-dump` (RobotCar, CMU, Nordland) và `dive-vpr-nb16a/b-msls` (MSLS). Notebook nằm ở `nb15_src/`. Chỉ cần chạy lại các kernel này khi muốn thêm descriptor hoặc dataset mới.

1b. 4Seasons descriptor dump + VIO (PUBLIC, mới 2026-10-29)

`nhatlong1103/vpr-dumps-4seasons`, link https://www.kaggle.com/datasets/nhatlong1103/vpr-dumps-4seasons (public, ≈ 3 GB, CC BY-NC-SA 4.0).

- **Nội dung:** 18 recording (7 scene) của 4Seasons, tổng 62,875 frame cam0, lấy mẫu sao cho hai frame cách nhau ≥ 1 m. Có 6 descriptor giống dump chính, kèm pose và **VIO thật** (odometry).
- **Tạo bởi:** `automation/make_dump4s.py` (7 kernel T4).
- **Lý do chỉ có 18 recording:** dataset ảnh `nhatlong1103/4seasons` chỉ có ảnh cho 21/30 recording; 3 recording không khớp được pose.
- **Scene có ≥ 2 recording, dùng được cho map + query:**
- `office_loop` (6 recording, 3.8 km);
- `countryside` (4, 6.5 km);
- `old_town` (4, 5.1 km; có `2021-05-10_21-32-00` là chiều tối).

```
dump/
  <desc>/4s_<scene>_<recording>.npy      desc ∈ megaloc, salad, mixvpr, cosplace, netvlad, dinov2 (float16, L2
  meta/4s_<scene>_<recording>.npz        names (timestamp ns), C (n,3) tâm camera trong h■ ENU riêng c■a scene
                                         R (n,3,3) camera→ENU (c■t 3 = h■■ng ■i), vio_C (n,3), vio_q (n,4) p
                                         (h■ riêng c■a recording, metric) = odometry, fq (ch■t l■■ng fusion G
README.md                                b■ng recording / s■ frame / chi■u dài
```

Quy ước toạ độ (đã verify): ECEF = T_e_gpsw · (scale · T_w_gpsw^{-1} · T_S_AS · p_GNSSPoses), đổi sang ENU tại pose đầu của recording đầu tiên của mỗi scene.

- Sai lệch giữa các recording của cùng scene (khoảng cách nearest-neighbor, trung vị): office_loop 0.34 m, countryside 0.49 m, old_town 1.2 m.
- Path VIO dài hơn path GNSS 1–12 %. Đó là drift thật của VIO, được giữ nguyên.

Loader: `V.load_4seasons(dump, desc, scene)`, gọi bằng `ds = "4s:<scene>"` trong `cert_seq.py`, `nav_demo.py` và `hmm_eval.py`.

- Map = recording sớm nhất của scene; query = các recording còn lại.
- Runner (`make_eval_kernel.py`) mount sẵn dataset này cùng dump chính (`DUMP_DS`) và tự gộp các thư mục `meta/`.

1c. Boreas descriptor dump + odometry bánh xe/gyro (PUBLIC, mới 2026-10-29)

Nguồn. Boreas (Burnett et al., IJRR 2023) trên S3 public `https://boreas.s3.amazonaws.com/`, tải không cần đăng nhập. Đây là tuyến Glen Shields (Toronto) dài khoảng 8 km, chạy nhiều lần từ 11/2020 đến 11/2021 (nắng, mưa, tuyết, chiều tối, đêm). Pose GT `applanix/camera_poses.csv` của mọi sequence nằm trong **một hệ ENU cố định chung**.

Dataset đã tạo (`automation/make_dump_boreas.py`: kernel GPU + internet, mỗi sequence khoảng 30 phút; `automation/upload_boreas.py`):

Slug	Nội dung
`nhatlong1103/boreas-vpr-dump`	31 sequence, 182,816 frame (≥ 1 m), 6 descriptor + meta
`nhatlong1103/boreas-vpr-images-1` … `-4`	JPEG 640 px của các frame đã chọn, chia 4 phần (tổng khoảng 13 GB)

```
dump/<desc>/boreas_<seq>.npy        desc ∈ megaloc, salad, mixvpr, cosplace, netvlad, dinov2 (float16, L2-nor
dump/meta/boreas_<seq>.npz          names (timestamp µs = tên ■nh), C (easting, northing, altitude; ENU chung
                                    rpy, vel, dmi (pulse encoder bánh xe), gyro_yaw (tích phân IMU yaw rate)
img/<seq>/<timestamp_us>.jpg        (các dataset boreas-vpr-images-k)
```

Loader: `V.load_boreas(dump, desc)`, gọi bằng `ds = "boreas"`.

- Map: `boreas-2020-11-26-13-58`.
- Query: 13 sequence đại diện (`V.BOREAS_Q`: tuyết 01/2021, chiều tối 04/2021, đêm 09/2021, hè, thu).
- Odometry thật (`cert_seq.odometry_wheel`): quãng đường = pulse × hệ số, hướng = gyro. Hệ số, dấu và hướng ban đầu được hiệu chỉnh trên 50 frame đầu của mỗi sequence.

Runner: `make_eval_kernel.py` mount sẵn `vpr-descriptor-dumps`, `vpr-dumps-4seasons` và `boreas-vpr-dump` (`DUMP_DS`).

2. Datasets ảnh gốc và trọng số (chỉ cần khi tạo dump mới)

Slug	Nội dung
`nhatlong1103/robotcar-seasons-v2`	RobotCar-Seasons v2: ảnh, `robotcar_v2_train.txt` (pose 4×4 phẳng, translation = index [3,7,11])
`nhatlong1103/extended-cmu-seasons` (+ `-2`)	Extended-CMU-Seasons, slice 2–25. Cấu trúc lồng `sliceN/sliceN/`; pose `camera-poses/*.txt`, xyz = 3 số cuối
`nhatlong1103/nordland-full-47-6gb`	Nordland đủ 4 mùa (35,768 ảnh/mùa, căn khung 1:1)
`ralphsitinh/msls-dataset`	Mapillary SLS (public, của người khác); dùng phần val CPH + SF
`nhatlong1103/4seasons` + `nhatlong1103/4seasons-poses`	4Seasons (ảnh 21/30 recording + pose 30 recording). Đã dump → §1b. `4seasons-poses` **public** từ 2026-10-29
`nhatlong1103/bundle`	Asset offline (public): `dino_salad.ckpt`, `dinov2_vitb14_pretrain.pth`, source salad/dinov2, wheels
`nhatlong1103/vpr-descriptor-weights`	Trọng số MixVPR, CosPlace, NetVLAD, MegaLoc (`automation/vpr_extra.py`). **Public** từ 2026-10-29, nên mọi acc chạy được kernel dump

BoQ vẫn chưa có trọng số (GitHub bị chặn, không có mirror).

Lưu ý vận hành khi tạo dump:

- Trên T4 phải dùng fp16. T4 chỉ giả lập bf16 nên chậm khoảng 7× (khoảng 3.5 ảnh/s cho 6 descriptor); `nb15_src/10_p0.src` đã sửa.
- Dataset `nhatlong1103/norlandshort` (Nordland 2 mùa dùng để smoke-test) đã có bản thay thế là `nordland-full-47-6gb` và dump. **Nên xoá; cần user tự xoá** vì môi trường chặn lệnh xoá.

3. Tài khoản Kaggle (8 acc × 2 T4 = 16 GPU chạy song song)

Token của mỗi acc nằm ở `~/.kaggle/tok_<username>` (chmod 600); repo public nên token không nằm trong git; lấy từ tin nhắn của user (thứ tự trùng bảng dưới). `~/.kaggle/access_token` là của acc 1. Các script `automation/make_eval_kernel.py`, `run_kernel.py` và `fanout.py` tự đọc đúng token theo tên acc, nên không cần `export`. Không ghi token vào `~/.bashrc`.

#	Username	Ghi chú
1	`nhatlong1103`	Chủ dump public và dataset ảnh gốc
2	`yuan1103`	

#	Username	Ghi chú
3	`synchan5704`	
4	`loinguyentran`	
5	`yeshenhi`	
6	`mucnokieu`	
7	`huyenco`	
8	`nguynduylm`	

Cả 8 acc đã được thử chạy GPU T4 thành công (2026-10-28); cả 8 token xác thực được với API (2026-10-29).

Quy tắc:

- Chia job cho cả 16 slot GPU trước.
- Chỉ dùng kernel CPU khi hết GPU và job đủ nhẹ.
- Thêm acc mới: lưu token vào `~/.kaggle/tok_<username>`. `fanout.py` tự nhận acc đó.

4. Chạy thí nghiệm trên Kaggle

Cách nhanh nhất là chia tự động ra mọi acc và mọi slot GPU, rồi tự gom kết quả vào `analysis/results/`:

```
python3 automation/fanout.py <work_dir> <slug_prefix> "eval_all.py rc salad;eval_all.py cmu salad;..." [n_ker
```

Chạy thủ công một kernel:

```
python3 automation/make_eval_kernel.py <kernel_dir> <slug> "eval_all.py rc salad;eval_all.py cmu salad" "" --
python3 automation/run_kernel.py <kernel_dir> <out_dir> 600 3 14400
cp <out_dir>/results/*.json analysis/results/
```

- Acc khác: `--owner <username>` (token tự đọc từ `~/.kaggle/tok_<username>`). Kernel tự mount dump public, không cần upload gì thêm.
- `make_eval_kernel.py` upload `analysis/*.py` thành dataset code `<acc>/dive-vpr-analysis`, rồi viết một kernel chạy song song các job. `--gpu` bật backend PyTorch (`SELFCAL_DEVICE=cuda`).
- Trên T4, `eval_all.py` mất khoảng 0.5–1 phút cho RobotCar/CMU/Nordland và lâu hơn cho MSLS.
- Kết quả: `results/*.json`, `logs/`, `figs/` (nếu có) trong output của kernel.
- Ngắt một kernel: chỉ `kaggle kernels delete -y <owner>/<slug>` mới có tác dụng.
- Chi tiết vận hành: `handoff/02_ENGINEERING_OPS_NOTES.md`.

5. Môi trường container (Claude Code cloud)

- Không có GPU, 4 nhân CPU. Chỉ dùng local CPU cho kiểm tra nhanh (dưới ~2 phút).
- Mọi tính toán nặng chạy trên Kaggle.
- Không chạy code bên thứ ba tải về trong container; chạy nó trong kernel Kaggle.